

```

/*:
## Exercise – Adopt Protocols: CustomStringConvertible, Equatable, and
Comparable

Create a `Human` class with two properties: `name` of type `String`, and
`age` of type `Int`. You'll need to create a memberwise initializer for the
class. Initialize two `Human` instances.
*/
class Human: CustomStringConvertible, Equatable, Comparable, Codable{

    var name: String
    var age: Int

    init(name: String, age: Int) {
        self.name = name
        self.age = age
    }

    var description: String {
        return "\(name) and \(age)"
    }

    static func == (lhs: Human, rhs: Human) -> Bool {
        return lhs.name == rhs.name && lhs.age == rhs.age
    }

    static func < (lhs: Human, rhs: Human) -> Bool{
        return lhs.age > rhs.age
    }
}

let human1 = Human(name: "Ryan", age: 18)
let human2 = Human(name: "Ethan", age: 17)

//: Make the `Human` class adopt the `CustomStringConvertible` protocol.
Print both of your previously initialized `Human` objects.

print(human1)
print(human2)

//: Make the `Human` class adopt the `Equatable` protocol. Two instances of
`Human` should be considered equal if their names and ages are identical to
one another. Print the result of a boolean expression evaluating whether or
not your two previously initialized `Human` objects are equal to eachother
(using `==`). Then print the result of a boolean expression evaluating
whether or not your two previously initialized `Human` objects are not equal
to eachother (using `!=`).
print(human1 == human2)

```

```
//: Make the `Human` class adopt the `Comparable` protocol. Sorting should be based on age. Create another three instances of a `Human`, then create an array called `people` of type `[Human]` with all of the `Human` objects that you have initialized. Create a new array called `sortedPeople` of type `[Human]` that is the `people` array sorted by age.
```

```
let humans = [human1, human2]  
let sortedPeople = humans.sorted(by: <)
```

```
//: Make the `Human` class adopt the `Codable` protocol. Create a `JSONEncoder` and use it to encode as data one of the `Human` objects you have initialized. Then use that `Data` object to initialize a `String` representing the data that is stored, and print it to the console.
```

```
import Foundation  
let jsonEncoder = JSONEncoder()  
if let jsonData = try? jsonEncoder.encode(human1), let jsonString =  
    String(data: jsonData, encoding: .utf8){  
    print(jsonString)  
}
```

```
/*:  
page 1 of 5 | [Next: App Exercise – Printable Workouts](@next)  
*/
```